Assignment:4

NAME: Nikita Sandip Patil
PRN NO:125B2F005
SUBJECT: DBMS
DIV: IT A4

Aim:

Write code to implement Procedure and Function.
(Employee Appraisal System with Outputs)

1. Stored Procedure

◆ **Definition**

A Stored Procedure is a collection of precompiled SQL statements stored in the database that can be executed whenever required.

◆ **Key Features**

1. Used to perform operations like INSERT, UPDATE, DELETE
2. Can accept input and output parameters
3. Improves performance (precompiled)
4. Reduces code repetition

Function

◆ **Definition**

A Function is a database object that performs a calculation and returns a single value.

◆ **Key Features**

1. Always returns a single value
2. Can be used inside SELECT statements
3. Used for calculations and data retrieval
4. Cannot modify database (generally)

- **Stored Procedures and display proper output after each operation in MySQL.**

Step 1: Create Table

```
CREATE TABLE Employee_Appraisal (
 emp_id INT PRIMARY KEY AUTO_INCREMENT,
 emp_name VARCHAR(50),
 salary INT,
 rating INT,
```

```
 bonus DECIMAL(10,2)
);
```

Output:
Query OK, 0 rows affected

Step 2: Insert Initial Data

```
INSERT INTO Employee_Appraisal (emp_name, salary, rating)
VALUES
('Nikita', 30000, 4),
('Vedant', 25000, 3),
('Amit', 35000, 5);
```

Output:
Query OK, 3 rows affected

Step 3: Display Table

```
SELECT * FROM Employee_Appraisal;
```

Output:

emp_id	emp_name	salary	rating	bonus
1	Nikita	30000	4	NULL
2	Vedant	25000	3	NULL
3	Amit	35000	5	NULL

Stored Procedure Operations

Step 4: Add Employee (Procedure)

```
CALL AddEmployee('Neha', 28000, 4);
```

Output:
Query OK, 1 row affected

Check Table After Insert

```
SELECT * FROM Employee_Appraisal;
```

Output:

emp_id	emp_name	salary	rating	bonus
1	Nikita	30000	4	NULL

2	Vedant	25000	3	NULL
3	Amit	35000	5	NULL
4	Neha	28000	4	NULL

Step 5: Update Salary (Procedure)

CALL UpdateSalary(1, 32000);

Output:
Query OK, 1 row affected

Check Table After Update

SELECT * FROM Employee_Appraisal;

Output:

emp_id	emp_name	salary	rating	bonus
1	Nikita	32000	4	NULL
2	Vedant	25000	3	NULL
3	Amit	35000	5	NULL
4	Neha	28000	4	NULL

Step 6: Delete Employee (Procedure)

CALL DeleteEmployee(3);

Output:
Query OK, 1 row affected

Check Table After Delete

SELECT * FROM Employee_Appraisal;

Output:

emp_id	emp_name	salary	rating	bonus
1	Nikita	32000	4	NULL
2	Vedant	25000	3	NULL
4	Neha	28000	4	NULL

Step 7: Show Employees (Procedure)

```
CALL ShowEmployees();
```

Output:

emp_id	emp_name	salary	rating	bonus
1	Nikita	32000	4	NULL
2	Vedant	25000	3	NULL
4	Neha	28000	4	NULL

---

2. Functions and Triggers in DBMS (Employee Appraisal System with Proper Outputs)

FUNCTIONS

Step 1: Create Function – Get Bonus

```
DELIMITER $$

CREATE FUNCTION GetBonus(e_id INT)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
DECLARE sal INT;
DECLARE rate INT;
DECLARE bonus DECIMAL(10,2);

SELECT salary, rating INTO sal, rate
FROM Employee_Appraisal
WHERE emp_id = e_id;

IF rate = 5 THEN
   SET bonus = sal * 0.20;
ELSEIF rate = 4 THEN
   SET bonus = sal * 0.15;
ELSE
   SET bonus = sal * 0.10;
END IF;

RETURN bonus;
END $$

DELIMITER ;
```

Output:

Query OK, 0 rows affected (0.02 sec)

Use Function

```
SELECT GetBonus(1);
```

Output:
GetBonus(1)
4800.00

Step 2: Create Function – Get Salary

```
DELIMITER $$

CREATE FUNCTION GetSalary(e_id INT)
RETURNS INT
DETERMINISTIC
BEGIN
DECLARE sal INT;

SELECT salary INTO sal
FROM Employee_Appraisal
WHERE emp_id = e_id;

RETURN sal;
END $$

DELIMITER ;
```

Output:
Query OK, 0 rows affected (0.01 sec)

Use Function

```
SELECT GetSalary(2);
```

Output:
GetSalary(2)
25000

Step 3: Create Function – Get Rating

```
DELIMITER $$

CREATE FUNCTION GetRating(e_id INT)
RETURNS INT
DETERMINISTIC
BEGIN
DECLARE r INT;

SELECT rating INTO r
FROM Employee_Appraisal
WHERE emp_id = e_id;

RETURN r;
END $$

DELIMITER ;
```

Output:
Query OK, 0 rows affected (0.01 sec)

Use Function

```
SELECT GetRating(1);
```

Output:
GetRating(1)
4

Step 4: Create Function – Total Employees

```
DELIMITER $$

CREATE FUNCTION TotalEmployees()
RETURNS INT
DETERMINISTIC
BEGIN
DECLARE total INT;

SELECT COUNT(*) INTO total FROM Employee_Appraisal;

RETURN total;
END $$
```

```
DELIMITER ;
```

Output:
Query OK, 0 rows affected (0.01 sec)

Use Function

```
SELECT TotalEmployees();
```

Output:

TotalEmployees()
3

TRIGGERS WITH PROPER OUTPUT

Step 5: BEFORE INSERT Trigger

```
DELIMITER $$

CREATE TRIGGER before_insert_employee
BEFORE INSERT ON Employee_Appraisal
FOR EACH ROW
BEGIN
IF NEW.rating = 5 THEN
   SET NEW.bonus = NEW.salary * 0.20;
ELSEIF NEW.rating = 4 THEN
   SET NEW.bonus = NEW.salary * 0.15;
ELSE
   SET NEW.bonus = NEW.salary * 0.10;
END IF;
END $$

DELIMITER ;
```

Output:
Query OK, 0 rows affected (0.01 sec)

Test INSERT Trigger

```
INSERT INTO Employee_Appraisal (emp_name, salary, rating)
VALUES ('Riya', 40000, 5);
```

Output:
Query OK, 1 row affected (0.01 sec)

Check Table
SELECT * FROM Employee_Appraisal;

Output:

emp_id	emp_name	salary	rating	bonus
1	Nikita	32000	4	NULL
2	Vedant	25000	3	NULL
4	Neha	28000	4	NULL
5	Riya	40000	5	8000.00

Step 6: BEFORE UPDATE Trigger

```
DELIMITER $$

CREATE TRIGGER before_update_employee
BEFORE UPDATE ON Employee_Appraisal
FOR EACH ROW
BEGIN
IF NEW.rating = 5 THEN
   SET NEW.bonus = NEW.salary * 0.20;
ELSEIF NEW.rating = 4 THEN
   SET NEW.bonus = NEW.salary * 0.15;
ELSE
   SET NEW.bonus = NEW.salary * 0.10;
END IF;
END $$

DELIMITER ;
```

Output:
Query OK, 0 rows affected (0.01 sec)

Test UPDATE Trigger

```
UPDATE Employee_Appraisal
SET salary = 50000
WHERE emp_id = 1;
```

Output:

Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1

Check Table

SELECT * FROM Employee_Appraisal;

Output:

emp_id	emp_name	salary	rating	bonus
1	Nikita	50000	4	7500.00
2	Vedant	25000	3	NULL
4	Neha	28000	4	NULL
5	Riya	40000	5	8000.00

Step 7: AFTER DELETE Trigger

```
CREATE TABLE Employee_Log (
log_id INT AUTO_INCREMENT PRIMARY KEY,
message VARCHAR(100)
);
```

Output:
Query OK, 0 rows affected (0.02 sec)

```
DELIMITER $$

CREATE TRIGGER after_delete_employee
AFTER DELETE ON Employee_Appraisal
FOR EACH ROW
BEGIN
INSERT INTO Employee_Log(message)
VALUES (CONCAT('Deleted employee: ', OLD.emp_name));
END $$

DELIMITER ;
```

Output:
Query OK, 0 rows affected (0.01 sec)

Test DELETE Trigger

```
DELETE FROM Employee_Appraisal
WHERE emp_id = 4;
```

Output:
Query OK, 1 row affected (0.01 sec)

Check Log Table

```
SELECT * FROM Employee_Log;
```

Output:

log_id	message
1	Deleted employee: Neha

Conclusion

The Employee Appraisal System was successfully implemented using **Stored Procedures, Functions, and Triggers** in MySQL. All database operations such as **insertion, updation, deletion, and retrieval** were performed efficiently using stored procedures. Functions were used to compute and return specific values like **bonus, salary, rating, and total number of employees**, ensuring better modularity and reusability.

Triggers were implemented to automate calculations and maintain data consistency by **automatically updating bonus values during INSERT and UPDATE operations**, and logging deleted records for tracking purposes. The outputs obtained after each step confirmed the correct working of all database objects.

Thus, this assignment demonstrates how advanced DBMS features improve **automation, accuracy, data integrity, and performance** in real-world applications.